Übungsblatt 3: Threads, Präemption und zeitgesteuertes Warten

Musterlösung · Echtzeitsysteme (EZS)

Prof. Dr.-Ing. Daniel Klünder

Musterlösung

Ziele und Ablauf

Sie übertragen das Aufgabenmodell aus Vorlesung 2 auf einen präemptiven FreeRTOS-Kernel: Zustände, Zuteilung, periodische Freigaben und Speicherbedarf. RTOS bedeutet Real-Time Operating System, Echtzeitbetriebssystem. Ein Thread ist ein fortsetzbarer Ablauf, ein Job eine Aktivierung. FreeRTOS nennt einen Thread Task. CMSIS-RTOS2 ist die Echtzeitkernel-Schnittstelle des Common Microcontroller Software Interface Standard.

Verbindlicher Übungskern ist die Klausurvorbereitung auf Papier. Die angegebenen Punkte dienen zur Selbsteinschätzung; die Modulnote entsteht durch die Papierklausur. Alle Rechen- und Analyseaufgaben sind mit den angegebenen Daten ohne Hardware oder erfolgreiche frühere Programmieraufgaben lösbar. Begründen Sie Entscheidungen und nennen Sie Modellannahmen.

Abschnitt	Minuten	Zweck
Papierkern mit Besprechung	50	32 Orientierungspunkte
Transfer und Besprechung	10	Rechnen und begründen
Praxisfall: vorbereitete Demonstration oder Papieralternative	15	Vorhersagen, erklären, Grenzen nennen

Der Übungsblock umfasst **75 Minuten**. Die Praxisphase umfasst etwa fünf Minuten Vorhersage, höchstens fünf Minuten Demonstration und fünf Minuten Erklärung. Ohne funktionierenden Aufbau bearbeiten Sie in denselben 15 Minuten den vollständig vorgegebenen Papierfall. **Nach spätestens drei Minuten technischer Fehlersuche wird zum Papierfall gewechselt.** Dadurch bleibt der Übungsblock auch bei Setup-Problemen nutzbar.

Für eine Demonstration verwendet die Lehrperson **STM32CubeMX**, **VS Code mit STM32 VS Code Extension** und **NUCLEO-H755ZI-Q / STM32H755ZIT6**. Sie bereitet die geprüften Projektvarianten vor dem Termin vor. Installation, Projektgenerierung, Integration, Regenerierungsprüfung, Build und Flash sind keine Pflichtaufgaben im Übungstermin. Eine eigene Implementierung ist freiwillige Vertiefung. Alle Zahlen dieses Blatts sind Lehrannahmen, soweit sie nicht ausdrücklich als eigene Messwerte gekennzeichnet sind.

Der Übungsblock ergänzt 85 Minuten Erklärung und 20 Minuten Quiz/Abschluss zum Termin mit 180 Minuten.

Teil A: Papieraufgaben

Aufgabe 1: Vom Callback zum Thread (6 Punkte)

1. Erklären Sie den Unterschied zwischen der endlichen `step()`-Funktion des Dispatchers aus Vorlesung 2 und der Eintrittsfunktion eines RTOS-Threads. Warum ist eine Endlosschleife im ersten Fall problematisch, im zweiten nicht grundsätzlich?
2. Ein Thread ruft `osDelay(5)` auf. Beschreiben Sie die Zustandsfolge bis zur erneuten Ausführung. Bedeutet das Ablaufen der Zeit automatisch, dass er läuft?
3. Heartbeat ruft stattdessen die übliche `HAL_Delay(...)` auf. Was verändert sich für die Prozessorzuteilung? Sind Interrupts dadurch automatisch gesperrt?

Notieren Sie Ihre Begründungen, nicht nur die Zustandsnamen.

Lösung und Bewertung: Je zwei Punkte pro begründeter Teilantwort.

1. Der Dispatcher erhält die Kontrolle erst nach Rückkehr aus `step()`; dessen Fortschritt wird ausdrücklich in Anwendungsdaten gespeichert. Ein nicht zurückkehrender Schritt verhindert weitere Callback-Aufrufe. Interrupts können trotzdem laufen. Ein RTOS-Thread besitzt dagegen einen gesicherten Fortsetzungszustand und eigenen Stack. Er kann durch einen Warteaufruf blockieren oder von einer höheren bereiten Task verdrängt werden. Eine hochpriore, dauerhaft bereite Endlosschleife bleibt trotzdem problematisch.
2. Running → Blocked beim Warteaufruf, Blocked → Ready nach Ablauf der fünf **Kernelticks**, Ready → Running erst nach Zuteilung. Eine höhere bereite Priorität kann den Start weiter verzögern.
3. `HAL_Delay` ist in der üblichen Hardware Abstraction Layer (HAL) aktives Warten. Heartbeat bleibt ausführungsbereit und verbraucht seine zugeteilte Zeit. Er wird dadurch nicht durch die Kernel-Warteverwaltung Blocked. Höhere Taskprioritäten können ihn weiterhin präemptieren, und erlaubte Interrupts können laufen.

Aufgabe 2: Zustände, Präemption und Fristen (10 Punkte)

Didaktisches **Ein-Kern-Modell**, keine Boardmessung:

Aufgabe	Priorität	Freigaben / Verhalten	CPU-Zeit pro Aktivierung
Ereignis-Task	höchste	wartet zunächst; Ereignis bei 2 ms	1 ms, danach erneut warten
Heartbeat	mittlere	periodisch bei 0, 4, 8 ms	1 ms, danach bis zur nächsten Freigabe warten
HeavyMath	niedrigste Anwenderpriorität	immer bereit	unbegrenzte Hintergrundarbeit

CPU bedeutet Central Processing Unit, zentrale Verarbeitungseinheit. Alle Wechsel-, Interrupt- und Warteverwaltungszeiten sind hier null. Der Ereignis-Thread ist bei $t = 0$ bereits wartend. Für Heartbeat gilt $D = 4\,\text{ms}$ als relative Deadline.

1. Zeichnen Sie die CPU-Belegung im halboffenen Intervall $[0, 10)\,\text{ms}$. Kennzeichnen Sie jeden Wechsel mit seiner Ursache. (4 Punkte)
2. Geben Sie die drei Taskzustände bei 1,5 und 2,5 ms an. (2 Punkte)

3. Berechnen Sie die insgesamt erhaltene CPU-Zeit jeder Aufgabe und die drei Heartbeat-Antwortzeiten. (2 Punkte)
4. Zusätzlich wird die Ereignis-Task bei 4 ms erneut bereit. Wann startet und endet der Heartbeat-Job dieser Freigabe, und hält er seine Deadline? (2 Punkte)

CPU-Belegung:

Intervall in ms	CPU	Ursache am Anfang
$[0, 1)$	Heartbeat	höchste bereite Priorität
$[1, 2)$	HeavyMath	Heartbeat blockiert
$[2, 3)$	Ereignis-Task	Ereignis macht höchste Priorität bereit
$[3, 4)$	HeavyMath	Ereignis-Task blockiert wieder
$[4, 5)$	Heartbeat	periodische Freigabe
$[5, 8)$	HeavyMath	Heartbeat blockiert
$[8, 9)$	Heartbeat	periodische Freigabe
$[9, 10)$	HeavyMath	Heartbeat blockiert

Bei 1,5 ms: Ereignis-Task Blocked, Heartbeat Blocked, HeavyMath Running. Bei 2,5 ms: Ereignis-Task Running, Heartbeat Blocked, HeavyMath Ready. Die verdrängte HeavyMath-Task wartet auf Zuteilung, nicht auf eine sachliche Wartebedingung.

Im ursprünglichen Modell erhält Heartbeat 3 ms, die Ereignis-Task 1 ms und HeavyMath 6 ms. Die Antwortzeiten $R = f - r$ sind jeweils 1 ms: Fertigstellung f minus Freigabe r.

Mit der zusätzlichen Ereignisfreigabe läuft die Ereignis-Task von 4 bis 5, Heartbeat von 5 bis 6. Somit $R = 6 - 4 = 2\,\text{ms} \leq D = 4\,\text{ms}$. Der Start ist verzögert, die Modellfrist trotzdem eingehalten. Bei $D = 1\,\text{ms}$ wäre sie verletzt. Diese Rechnung gilt nur unter den genannten Modellannahmen.

Aufgabe 3: Ticks, Drift und ein verpasstes Ziel (8 Punkte)

1. Ein Kernel läuft mit $f_{\text{tick}} = 100\,\text{Hz}$. Bestimmen Sie Tickperiode und aufgerundete Tickzahl für 5 ms und 500 ms. Warum bedeutet `osDelay(1)` nicht eine garantierte Mindestwartezeit von 10 ms? (3 Punkte)
2. Anderes Modell: Tickgrenzen in 1-ms-Abständen, Arbeitsdauern nacheinander 2, 4 und 1 ms; der erste Start liegt bei null. Alle Arbeitsenden liegen genau auf Tickgrenzen, weitere Störungen fehlen. Berechnen Sie die nächsten drei Starts bei „Arbeit, dann `osDelay(10)`“ und bei einem absoluten 10-Tick-Sollraster. (3 Punkte)
3. Ein Sollziel liegt bei Tick 20, die Arbeit endet erst bei Tick 23. Nennen Sie eine sinnvolle Reaktion mit Periode 10 Ticks und erklären Sie, warum nicht sofort mehrere LED-Umschaltungen nachgeholt werden sollten. (2 Punkte)

1. $T_{\text{tick}} = 1/100\,\text{s} = 10\,\text{ms}$. Für t Millisekunden gilt $n = \lceil t f_{\text{tick}}/1000 \rceil$. Fünf Millisekunden ergeben $\lceil 0{,}5 \rceil = 1$ Tick, 500 Millisekunden 50 Ticks. Erfolgt `osDelay(1)` eine Millisekunde vor der nächsten Tickgrenze, wird der Thread schon nach ungefähr einer Millisekunde Ready. Die Tickphase bestimmt den Rest; höhere Last kann den anschließenden Start wieder verzögern.
2. Relative Starts: $0 + 2 + 10 = 12$, $12 + 4 + 10 = 26$, $26 + 1 + 10 = 37\,\text{ms}$. Absolute Starts im Modell: 10, 20, 30 ms. Die Arbeit passt jeweils vor den nächsten Sollpunkt. Das absolute Raster addiert die Arbeitsdauer nicht zur Periode. Es garantiert unter Störungen nicht den tatsächlichen Start.

3. Laborpolitik: Zielverfehlung zählen, neues Ziel $23+10=33$ setzen und bis dahin blockieren. Ein direktes mehrfaches Nachholen würde Ereignisse künstlich zusammenballen und die Last verschärfen. Alternativ kann ein spezifiziertes System veraltete Aktivierungen überspringen oder einen Fehler melden. Der Warteziel-Zähler ist ohne zusätzliche Deadlinedefinition kein allgemeiner Deadline-Miss-Zähler.

Aufgabe 4: Speicher, Prioritäten und Ereignisse (8 Punkte)

1. CMSIS-Stacks 512, 1024 und 1024 Byte; angenommener Verwaltungsblock (TCB, Task Control Block) jeweils 80 Byte. Berechnen Sie den Bedarf dieser drei Stack-/TCB-Paare. Warum reicht diese Rechnung noch nicht als Heapnachweis? Wie viele Byte sind 128 native `StackType_t`-Elemente zu je vier Byte? (3 Punkte)
2. HeavyMath wird über Heartbeat priorisiert und bleibt dauerhaft Ready. Heartbeat ist ebenfalls Ready. Hilft `osThreadYield()` in HeavyMath? Ist das Priority Inversion? (3 Punkte)
3. Die logische Interruptschwelle für Kernelaufrufe sei 5, mit vier implementierten Prioritätsbits und passender Gruppierung. Darf ein IRQ mit logischer Priorität 3 `osThreadFlagsSet` verwenden? Darf ein erlaubter IRQ `osThreadFlagsWait` verwenden? Zählt zweimaliges Setzen desselben Bits vor der Verarbeitung zwei Ereignisse? (2 Punkte)

1. $512 + 1024 + 1024 + 3 \cdot 80 = 2800$ Byte. Idle, optionale Timer-Task, weitere Objekte, Speicherzuschläge und Ausrichtung fehlen. Die tatsächliche TCB-Größe ist versions- und konfigurationsabhängig. Der Handlerstack und übrige Daten müssen ebenfalls in die Gesamtspeicherbilanz. Bereits im Heap enthaltene Taskblöcke werden nicht zusätzlich außerhalb dieses Heaps gezählt. $128 \cdot 4 = 512$ Byte; CMSIS `stack_size=512` ist dagegen bereits Byte.
2. Yield fordert Auswahl an, macht HeavyMath aber nicht Blocked. Die höchste bereite Priorität bleibt HeavyMath. Heartbeat verhungert: Starvation. Es gibt keine gemeinsame Ressource und keine Warteabhängigkeit zu einem niedrigen Ressourcenbesitzer; daher keine Priority Inversion.
3. Logische Priorität 3 ist dringlicher als 5 und darf in dieser Konfiguration keine Kernel-API verwenden. Auch ein erlaubter IRQ darf nicht mit `osThreadFlagsWait` blockieren; er darf das passende Bit mit `osThreadFlagsSet` setzen. Ein Bit bleibt bei zweimaligem Setzen vor dem Wait nur gesetzt und zählt nicht verlustfrei zwei Ereignisse.

Transfer: Eine Diagnose braucht Zustände · 10 Minuten

Zusätzliche Klausurübung ohne weitere Punkte: Eine Heartbeat-LED steht still. Drei Erklärungen werden angeboten: fehlender GPIO-Takt, fehlgeschlagene Threadanlage und Starvation durch höhere dauerhaft bereite Arbeit.

1. Nennen Sie zu jeder Erklärung einen passenden Prüfpunkt. Warum folgt aus derselben sichtbaren LED allein keine eindeutige Diagnose?
2. Im Modell sind GPIO und Kernelstart geprüft, das Heartbeat-Handle gültig und Heartbeat Ready. Eine höhere Task bleibt Ready. Welche Erklärung passt jetzt? Kann die höhere Task durch `osThreadYield()` Platz für Heartbeat schaffen?

Musterlösung Transfer 3

GPIO: tatsächliche Initialisierung, Takt und erreichter Schreibzugriff prüfen. Threadanlage: gültigen Rückgabewert und Fehlerpfad prüfen. Starvation: tatsächliche Prioritäten, Ready-Zustände und Zuteilung prüfen. Ein unveränderter Ausgang unterscheidet diese Ursachen nicht.

Unter den zusätzlichen Voraussetzungen passt Starvation: Die höchste bereite Priorität bleibt höher als Heartbeat. Yield blockiert sie nicht. Echte Wartebedingungen oder eine begründete Prioritäts-/Laständerung können Heartbeat Auswahl ermöglichen. Eine geteilte Ressource wird nicht vorausgesetzt; daher ist dies keine Priority Inversion.

Teil B: Praxisfall · 15 Minuten

Praxisfall 3: Yield oder echtes Warten?

Papieralternative, Ein-Kern-Lehrmodell: Ereignis-Task hat höchste, HeavyMath mittlere und Heartbeat niedrigste Anwenderpriorität. Heartbeat ist bei null Ready, arbeitet einmal 1 ms und wartet danach. Seine relative Deadline ist 5 ms. Die Ereignis-Task wartet bis zu einem Ereignis bei 5 ms, arbeitet dann 1 ms und wartet erneut. Kosten der API und Wechsel sind null.

HeavyMath führt wiederholt 2 ms Arbeit aus. Variante Y ruft danach `osThreadYield()` auf und bleibt Ready. Variante W ruft danach `osDelay(4)` auf. Für Variante W gelten ein 1-kHz-Tick, Aufruf genau auf Tickgrenzen und keine weiteren Verzögerungen. Der Kernel verwendet feste Prioritäten und Präemption.

1. Zeichnen Sie die CPU-Belegung von 0 bis 6 ms für beide Varianten. Wird Heartbeat rechtzeitig fertig?
2. Welche Zustandsänderung macht den Unterschied? Ist die Verzögerung eine Priority Inversion?

Bei vorbereitetem Board: Die Lehrperson zeigt den vorbereiteten Normal- und Starvation-Lauf des Moduls `lab03_rtos`. Die Papier-Varianten Y/W erklären anschließend die Abhilfe; ihre exakten Modellzeiten gehören nicht zum Boardversuch.

Musterlösung Praxisfall 3

Y: HeavyMath 0–5 ms, Ereignis-Task 5–6 ms. Yield bei 2 und 4 ms lässt HeavyMath als höchste bereite Task übrig. Heartbeat erhält bis zur Deadline bei 5 ms keine Zuteilung und verfehlt sie.

W: HeavyMath 0–2, Heartbeat 2–3, Idle 3–5, Ereignis-Task 5–6 ms. HeavyMath ist nach dem Wait bis Tick 6 Blocked. Heartbeat endet bei 3 ms und hält die Frist. Nach 6 ms darf HeavyMath erneut ausgewählt werden. Es gibt keine Ressourcenabhängigkeit zu einem niedrigen Besitzer: Das Problem ist Starvation, keine Priority Inversion.

Ergebnis und freiwillige Vertiefung

Halten Sie die begründeten Papierlösungen (32 Orientierungspunkte), Ihre Transferantworten und die Erklärung des Praxisfalls fest. Projektdateien, Versionslisten, Flashnachweise und Messprotokolle sind keine Pflichtabgabe dieses Übungsblocks. Für das Verständnis des nächsten Blatts ist keine erfolgreiche Firmwareintegration erforderlich.

Die freiwillige Programmiervertiefung enthält die bisherige technische Vorbereitung und Integration. Sie richtet sich an die Lehrperson sowie an Studierende, die außerhalb des Termins

mehr Praxis wünschen. Ein fehlender Boardlauf beeinträchtigt die Bearbeitbarkeit der Papieraufgaben nicht. In eigenen Protokollen bleiben ungemessene Zeitfelder leer; beobachtete Stichprobenmaxima sind keine allgemeine Worst-Case-Garantie.

Quellen: Vorlesung 3; Arm CMSIS-RTOS2-Dokumentation zu Wait, Threads und Thread Flags; FreeRTOS-Dokumentation zum Cortex-M-Port; ST UM2408, RM0399 und AN5361; Dokumentation der installierten STM32 VS Code Extension. Ihre Projektversionen haben Vorrang vor Annahmen über den aktuellen öffentlichen Entwicklungszweig.